

JSON Engine

[2. Operationen \(DSL op\)](#)

[2.1. ASSERT \(Validierung\)](#)

[Erweiterte JSONPath-Syntax:](#)

[3. Referenz der JSON-Parameter](#)

[Praxis-Tutorial: JSON-Engine Demonstration](#)

[Vollständiges YAML-Szenario](#)

[Log-Output der Engine](#)

[Troubleshooting & Best Practices](#)

[Troubleshooting-Checkliste](#)

[Performance-Hinweise](#)

Die JSON-Engine ermöglicht die gezielte Validierung von Inhalten innerhalb von JSON-Dateien. Sie unterstützt das direkte Auslesen tief verschachtelter Strukturen mittels einer erweiterten JSONPath-Syntax und führt flexible Text- sowie reguläre Vergleiche gegen erwartete Werte durch.

1. JSON Engine - Kompakte System-Referenz für KI-Modelle

1 | YAML

```
1 # AGATE JSON-Engine Syntax Referenz
2 # Nutze dies für Testfall-Generierung und Analyse
3
4 - type: JSON
5   op: ASSERT
6   file: "..."/>

```

2. Operationen (DSL op)

Die JSON-Engine konzentriert sich vollends auf die präzise Validierung bestehender Strukturen. Sie liest die gesamte Zieldatei in den Speicher, bevor die Validierungsschritte evaluiert werden.

2.1. ASSERT (Validierung)

Prüft eine oder mehrere Positionen innerhalb einer JSON-Datei ab. Im Gegensatz zu sequenziellen Asserts führt diese Engine **alle** definierten Prüfungen innerhalb des Schritts aus, sammelt eventuelle Fehler und wirft erst am Ende des Schritts einen zusammenfassenden Fehler (Sammel-Validierung).

Parameter	Beschreibung	Erforderlich
-----------	--------------	--------------

file	Der Pfad zur JSON-Datei, die validiert werden soll.	Ja
parameters	Ein Set aus sequenziell nummerierten Schlüssel-Wert-Paaren (<code>path1 / value1</code> , <code>path2 / value2</code> , ...).	Ja
condition	<i>Optional:</i> Bedingung für die Ausführung des gesamten ASSERT-Schritts.	Nein

Erweiterte JSONPath-Syntax:

Die Engine nutzt einen optimierten Tokenizer (`getValueByPath`), der folgende Navigations-Muster auflöst:

- **Standard-Objekte:** Dot-Notation zur Feldnavigation (z.B. `$.logs.level1`).
- **Unbegrenzt verschachtelte Arrays:** Direkt aufeinanderfolgende Indizes (z.B. `$.logs[0][10]`).
- **Komplexe Schlüssel:** Sonderzeichen oder rein numerische Keys in eckigen Klammern mit einfachen oder doppelten Anführungszeichen (z.B. `$['event.code']` oder `$["10"]`).
- **Root-Präfix:** Das führende `$` sowie darauffolgende Punkte (z.B. `$.`) sind optional und werden automatisch normalisiert.

3. Referenz der JSON-Parameter

Hier sind alle Parameter für den JSON-Schritt im Überblick:

Schritt	Parameter	Beschreibung
ASSERT	file	Der Pfad zur JSON-Datei auf dem Dateisystem.
	parameters	Container für die paarweisen Pfade (<code>pathX</code>) und

		Erwartungswerte (valueX).
	condition	<i>Optional:</i> Bedingung für die Ausführung des gesamten Schritts.

Praxis-Tutorial: JSON-Engine Demonstration

Dieses Beispiel demonstriert, wie komplexe JSON-Strukturen, verschachtelte Arrays und dynamische Platzhalter in einem einzigen Validierungsblock geprüft werden.

Vollständiges YAML-Szenario

```

1 | YAML
1  testCases:
2    - id: "TC_JSON_VALIDATION"
3      description: "Umfassende Validierung einer API-Response-Datei"
4      variables:
5        expected_status: "SUCCESS"
6        environment: "PROD"
7      steps:
8        # 1. Multi-Assert auf eine generierte JSON-Datei
9        - type: JSON
10         op: ASSERT
11         file: "C:\\tmp\\api_response.json"
12         parameters:
13           # Standard Feld-Validierung mit Variablen-Auflösung
14           path1: "$.status"
15           value1: "{B[expected_status]}"
16
17           # Komplexe Keys mit Sonderzeichen via Bracket-Notation
18           path2: "$['metadata.info']['env.type']"
19           value2: "{B[environment]}"
20
21           # Verschachteltes Array (Erstes Element, Feld 'id')
22           path3: "$.data.users[0].id"
23           value3: "1001"
24
25           # Wildcard / Regex-Vergleich (Wert muss mit 'Session-' beginn
26           path4: "$.data.session_token"
27           value4: "Session-.*"
28

```

Log-Output der Engine

Während der Ausführung gibt die Engine den Status jeder einzelnen Pfad-Evaluierung in Echtzeit aus. Bei Erfolg wird der gesamte Block positiv abgeschlossen:

```

1 | Plaintext
1  Checking JSON assertions for file: C:\tmp\api_response.json
2  [PASSED] $.status == "SUCCESS"
3  [PASSED] $['metadata.info']['env.type'] == "PROD"
4  [PASSED] $.data.users[0].id == "1001"
5  [PASSED] $.data.session_token == "Session-.*"

```

Troubleshooting & Best Practices

Falls Validierungen fehlschlagen oder die Engine unerwartet abbricht, nutze diese Checkliste zur Fehlerbehebung:

Troubleshooting-Checkliste

- **Test bricht sofort mit `RuntimeException` ab (Infrastruktur-Fehler):**

- *Meldung:* `JSON ASSERT operation requires 'file' parameter...` oder `JSON file not found....`
- *Ursache:* Der Pfad fehlt oder die Datei wurde im vorherigen Testverlauf (z.B. durch ein CMD-Skript) nicht korrekt im angegebenen Verzeichnis abgelegt.

- **Mehrere Felder schlagen fehl (Sammel-Meldung):**

- Wenn `path1` fehlschlägt, bricht die Engine **nicht** ab. Sie prüft `path2` und alle weiteren Werte bis zum Schluss durch. Sind Fehler aufgetreten, wirft sie eine zusammenfassende `RuntimeException` mit allen fehlgeschlagenen Pfaden im Detail:

```
1 | Plaintext
```

```
1 | RuntimeException: JSON Assertion failed for file: C:\tmp\api_response.json
2 |   - Path [$.status] failed! Expected: [SUCCESS], Actual: [FAILED]
3 |   - Path [$.data.users[0].id] failed! Expected: [1001], Actual: [9999]
4 |
```

- **Absturz während der Pfad-Evaluierung (`crashed during evaluation`):**

- *Ursache:* Der `JSONPath` versucht eine Operation auf einem falschen Datentyp auszuführen. Wenn im Pfad `$.users[0]` definiert ist, an dieser Stelle im JSON jedoch ein Objekt statt eines Arrays steht, protokolliert die Engine das im Log (`0 0getauscht JSON Niz za indeks [0], ali pronaden: ...`) und markiert den Pfad als *failed*.

- **Verwendung von Wildcards (`*`):**

- Enthält das `valueX` -Feld ein `*` -Zeichen, schaltet die Engine intern automatisch auf eine RegEx-Prüfung (`actualResolved.matches(expectedResolved)`) um. Stelle in diesem Fall sicher, dass der restliche String valider Regex-Syntax entspricht.

Performance-Hinweise

- **Datei-Caching:** Da die Engine pro Schritt den gesamten Dateiinhalt via `Files.readAllBytes` einliest, empfiehlt es sich bei sehr großen JSON-Strukturen (mehrere Megabyte), möglichst viele Pfad-Validierungen in **einem einzigen** `ASSERT` -Schritt mit fortlaufenden Nummern (`path1` , `path2` ...) zu bündeln, anstatt mehrere getrennte JSON-Schritte nacheinander auf dieselbe Datei anzusetzen.